

UNA BARREIRO — CENTRO UNIVERSITÁRIO

Disciplina: Gestão e Qualidade de Software

Professor: Daniel Henrique Matos de Paiva

LISTA DE EXERCÍCIOS III — DOCUMENTO DE ESPECIFICAÇÃO TÉCNICA

ParkSim

Sistema Simulador de Estacionamento Inteligente com Banco de Dados em Nuvem

EQUIPE DE DESENVOLVIMENTO / INTEGRANTES

João Vitor — RA: 32513480

Rafael Luiz Ferreira de Souza — RA: 32511503

Pietro Cardoso de Oliveira — RA: 32515280

Repositório: github.com/Lebot007/gqs-algoritmo-03-py

SUMÁRIO DO DOCUMENTO

1. Visão Geral do Sistema

- 1.1 Escopo do Produto
- 1.2 Objetivo do Sistema
- 1.3 Público-Alvo

2. Matriz de Requisitos

- 2.1 Requisitos Funcionais (RF01 a RF08)
- 2.2 Requisitos Não Funcionais — ISO/IEC 25010 (RNF01 a RNF06)

3. Modelagem de Dados e Arquitetura

- 3.1 Modelo Conceitual e DER Textual
- 3.2 Esquema SQL Real do Projeto (PostgreSQL / Supabase)
- 3.3 Diagrama de Classes e Módulos JavaScript

4. Estratégia de Versionamento e Qualidade

- 4.1 Controle de Versão e GitHub Flow
- 4.2 Convenção de Commits (Conventional Commits)
- 4.3 Política de Pull Requests e Code Review
- 4.4 Documentação do Repositório (README.md)

5. Plano de Garantia de Qualidade (QA)

- 5.1 Matriz de Cenários de Testes Integrados (CT01 a CT04)

6. Bônus: Roteiro da Apresentação Técnica (Pitch de 9 Slides)

1. Visão Geral do Sistema

1.1 Escopo do Produto

O **ParkSim** é uma aplicação web interativa em arquitetura Single Page Application (SPA), desenvolvida com tecnologias web fundamentais nativas (HTML5, CSS3 com variáveis e animações fluídas, e JavaScript puro estruturado em ES Modules nativos), sem a utilização de frameworks ou bibliotecas pesadas de interface (como React, Vue ou Angular).

O sistema simula o ciclo de vida completo de um estacionamento inteligente composto por **8 vagas** numeradas de forma individual. Veículos gerados sinteticamente com placas no padrão oficial Mercosul (formato `LLLNLNN`) e diversas opções de cores entram no pátio, realizam manobras de tráfego pela pista de circulação, estacionam em vagas disponíveis e deixam o pátio após um período de permanência dinamicamente calculado.

Todo o estado operacional e o histórico das movimentações são sincronizados em tempo real com uma instância de banco de dados relacional **PostgreSQL na nuvem Supabase**, garantindo persistência durável, execução de regras de negócio via triggers em PL/pgSQL e controle rigoroso de acesso por políticas de segurança no nível de linha (*Row Level Security - RLS*).

1.2 Objetivo do Sistema

O objetivo primário do ParkSim é servir como um modelo prático de demonstração técnica e acadêmica para a disciplina de **Gestão e Qualidade de Software**. O projeto demonstra a viabilidade de construir um software de alta performance,

visualmente atraente a 60 FPS e totalmente integrado a uma infraestrutura BaaS (*Backend-as-a-Service*) em nuvem, aplicando rigorosos conceitos de engenharia de software, tais como:

- Separação estrita de responsabilidades em arquitetura modular (DAO, UI, Motor de Simulação e Orquestrador).
- Transferência de regras de cálculo de negócio do client-side para o banco de dados via `BEFORE UPDATE TRIGGER` em PL/pgSQL.
- Segurança por princípio do menor privilégio através de `Row Level Security` e autenticação baseada em JWT (Supabase Auth).
- Geração automatizada de relatórios executivos em formato PDF cliente-side via jsPDF.

1.3 Público-Alvo

O sistema foi concebido para atender a três perfis distintos de usuários:

- **Avaliadores Acadêmicos e Docentes:** Professores e bancas examinadoras que buscam auditar a aplicação de conceitos teóricos de qualidade de software, modelagem de dados e arquitetura relacional.
- **Administradores e Gestores do Estacionamento:** Operadores autenticados que demandam um painel gerencial em tempo real para monitorar ocupação, tempo médio de permanência, volume total de saídas e exportação de relatórios auditáveis.
- **Visitantes / Usuários Finais:** Observadores públicos que interagem com a simulação visual sem permissão para acessar os dados sensíveis e relatórios administrativos fechados.

2. Matriz de Requisitos

A matriz de requisitos estabelece os compromissos funcionais e os atributos de qualidade exigidos pelo sistema ParkSim, mapeados individualmente com critérios de aceite testáveis.

2.1 Requisitos Funcionais (RF)

ID	TIPO	DESCRIÇÃO	PRIORIDADE	CRITÉRIO DE ACEITE
RF01	RF	Simular entrada automatizada de veículos com placa Mercosul e cor aleatória.	ALTA	Gerar automaticamente placa no formato regulamentar <code>LLLNLNN</code> e cor aleatória entre 6 opções (azul, vermelho, preto, branco, prata, verde), alocando o carro em uma vaga livre (1 a 8).
RF02	RF	Registrar a entrada do veículo no banco de dados remoto com timestamp.	ALTA	Ao entrar no pátio, disparar <code>INSERT</code> na tabela <code>movimentacoes</code> com <code>hora_entrada = now()</code> , <code>status ESTACIONADO</code> , <code>placa</code> , <code>cor</code> , <code>vaga</code> e <code>número da sessão</code> .
RF03	RF	Registrar saída do veículo e calcular o tempo de permanência via Trigger PL/pgSQL.	ALTA	Ao sair do pátio, disparar <code>UPDATE</code> preenchendo <code>hora_saida = now()</code> e <code>status SAIU</code> . A trigger <code>trg_calcular_tempo</code> no banco deve preencher obrigatoriamente <code>tempo_minutos</code> .

ID	TIPO	DESCRIÇÃO	PRIORIDADE	CRITÉRIO DE ACEITE
RF04	RF	Exibir Painel Administrativo em tempo real com métricas e tabela de movimentações.	ALTA	O painel deve conter 4 cards contadores (Total Registrados, Estacionados, Saídas, Tempo Médio) e tabela limitada visualmente às 50 movimentações mais recentes.
RF05	RF	Restringir a visualização do painel administrativo a usuários autenticados (Admin).	ALTA	Visitantes sem login veem o painel bloqueado com cadeado (🔒). Apenas usuários logados via Supabase Auth executam o <code>SELECT</code> e visualizam o conteúdo.
RF06	RF	Disponibilizar controles de execução (Iniciar, Pausar, Reiniciar) e seletor de velocidade (1x, 2x, 3x).	ALTA	Os botões Iniciar/Pausar/Reiniciar devem alterar o estado do simulador; o seletor 1x/2x/3x deve acelerar proporcionalmente os ticks, tempos de spawn, permanência e animações visuais.
RF07	RF	Exportar relatório executivo e acadêmico em formato PDF.	MÉDIA	Administrador autenticado pode clicar em "Exportar PDF", gerando via jsPDF o documento com estatísticas da sessão, tabela detalhada e seção de conceitos teóricos de banco de dados.
RF08	RF	Identificar visualmente placas, cores e status nas linhas da tabela do painel.	BAIXA	A tabela deve conter pílulas estilizadas para os status "Estacionado" (verde) e "Saiu" (azul), além de indicadores visuais das cores dos veículos.

2.2 Requisitos Não Funcionais (RNF) — ISO/IEC 25010

ID	TIPO	CATEGORIA ISO 25010	DESCRIÇÃO	CRITÉRIO DE ACEITE
RNF01	RNF	Usabilidade / Adequação do Layout	Interface com suporte responsivo amplo em dispositivos móveis e desktops.	Layout totalmente adaptável em resoluções de 320px (mobile) até 1440px+ (desktop HD/4K) com tema dark moderno e sem rolagem horizontal indesejada.
RNF02	RNF	Eficiência de Desempenho	Taxa de quadros estável em 60 FPS durante a execução da simulação.	Animações CSS/WAAPI baseadas exclusivamente nas propriedades <code>transform</code> e <code>opacity</code> acionadas via GPU, garantindo renderização fluída sem travamentos no DOM.
RNF03	RNF	Segurança de Acesso	Políticas de segurança em nível de linha (Row Level Security - RLS) no banco.	A tabela <code>movimentacoes</code> deve restringir a operação de consulta <code>SELECT</code> exclusivamente à role <code>authenticated</code> , permitindo <code>INSERT/UPDATE/DELETE</code> para o fluxo do simulador.
RNF04	RNF	Disponibilidade	Infraestrutura de banco de dados e autenticação hospedada em nuvem.	Operação sobre a infraestrutura em nuvem do Supabase (PostgreSQL) com meta de SLA de disponibilidade de 99,9% e reconexão graciosa em caso de interrupção de rede.

ID	TIPO	CATEGORIA ISO 25010	DESCRIÇÃO	CRITÉRIO DE ACEITE
RNF05	RNF	Manutenibilidade	Arquitetura de software modularizada e código limpo sem acoplamento.	Código JavaScript dividido rigidamente em 6 módulos ES com responsabilidades únicas e sem dependência de bibliotecas ou frameworks externos de terceiros no core.
RNF06	RNF	Conformidade Legal / LGPD	Conformidade total com a Lei Geral de Proteção de Dados (Lei nº 13.709/2018).	O sistema armazena unicamente dados sintéticos simulados (placas aleatórias, cores, números de vaga e timestamps), sem coletar ou processar nenhum Dado Pessoal Identificável (PII).

3. Modelagem de Dados e Arquitetura

3.1 Modelo Conceitual e DER Textual

O modelo de dados do ParkSim foi desenhado seguindo a primeira forma normal (1FN) e focado na rastreabilidade temporal rigorosa dos veículos no pátio de estacionamento.

Estrutura da Entidade Principal: movimentacoes

- `id` (PK, BIGINT, Identity): Chave primária gerada automaticamente pelo banco PostgreSQL.
- `sessao` (INT, NOT NULL): Número identificador da sessão ativa da simulação (incrementado a cada reinicialização).
- `placa` (TEXT, NOT NULL): Identificador sintético do veículo no padrão Mercosul (ex: ABC1D23).
- `cor` (TEXT, NOT NULL): Identificação da cor estilizada (ex: azul , vermelho , preto , branco , prata , verde).
- `vaga` (INT, NOT NULL): Número do box/vaga ocupado no pátio (intervalo de 1 a 8).
- `hora_entrada` (TIMESTAMPTZ, NOT NULL): Registro de data e hora exatas com fuso horário da entrada do veículo no pátio (default `now()`).
- `hora_saida` (TIMESTAMPTZ, NULL): Registro de data e hora exatas da saída (permanece nulo enquanto o veículo estiver estacionado).
- `tempo_minutos` (INT, NULL): Tempo total de permanência calculado em minutos (preenchido exclusivamente via Trigger no banco).
- `status` (TEXT, NOT NULL): Estado operacional atual do registro ('ESTACIONADO' ou 'SAIU').

3.2 Esquema SQL Real do Projeto (Supabase / PostgreSQL)

Abaixo é apresentado o script DDL e DCL exato, testado e em produção na instância do Supabase PostgreSQL:

```
-- 1. Criação da tabela principal de movimentações
CREATE TABLE public.movimentacoes (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  sessao INT NOT NULL DEFAULT 1,
  placa TEXT NOT NULL,
  cor TEXT NOT NULL,
  vaga INT NOT NULL,
  hora_entrada TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```

    hora_saida TIMESTAMPTZ,
    tempo_minutos INT,
    status TEXT NOT NULL DEFAULT 'ESTACIONADO'
);

-- 2. Função em PL/pgSQL para cálculo de permanência
CREATE OR REPLACE FUNCTION public.calcular_tempo()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.hora_saida IS NOT NULL THEN
        NEW.tempo_minutos := GREATEST(1,
            ROUND(EXTRACT(EPOCH FROM (NEW.hora_saida - NEW.hora_entrada)) / 60)::INT
        );
    END IF;
    RETURN NEW;
END; $$;

-- 3. Trigger associada à tabela de movimentações (BEFORE UPDATE)
CREATE TRIGGER trg_calcular_tempo
BEFORE UPDATE ON public.movimentacoes
FOR EACH ROW EXECUTE FUNCTION public.calcular_tempo();

-- 4. Habilitação de Row Level Security (RLS)
ALTER TABLE public.movimentacoes ENABLE ROW LEVEL SECURITY;

-- 5. Políticas de Acesso RLS (DCL)
CREATE POLICY insert_publico ON public.movimentacoes FOR INSERT
TO anon, authenticated WITH CHECK (true);

CREATE POLICY update_publico ON public.movimentacoes FOR UPDATE
TO anon, authenticated USING (true) WITH CHECK (true);

CREATE POLICY delete_publico ON public.movimentacoes FOR DELETE
TO anon, authenticated USING (true);

CREATE POLICY select_admin ON public.movimentacoes FOR SELECT
TO authenticated USING (true);

```

3.3 Diagrama de Classes e Módulos JavaScript

A arquitetura client-side do ParkSim é composta por 6 módulos ES nativos, organizados com alto nível de coesão e baixo acoplamento:

MÓDULO / ARQUIVO	ATRIBUTOS / EXPORTAÇÕES
js/config.js	CONFIG, SUPABASE_URL, SUPABASE_ANON_KEY
js/supabase.js	supabase

MÓDULO / ARQUIVO	ATRIBUTOS / EXPORTAÇÕES
js/banco.js	getSessaoAtual , novaSessao , resetarSessao , limparMovimentacoes , login , logout , sessaoAtiva
js/ simulacao.js	criarSimulacao (retorna métodos de controle: iniciar , pausar , reiniciar , setVelocidade , estaRodando)
js/ui.js	agendar , cancelarAgendamentos , formatarRelogio , formatarHora , atualizarTimer , setStatus , se exportarPDF
js/app.js	simulacao , tentarLogin , refrescarRelatorio , aplicarAuth , iniciarApp

4. Estratégia de Versionamento e Qualidade

4.1 Controle de Versão e GitHub Flow

O gerenciamento de código-fonte do projeto utiliza a plataforma GitHub no repositório oficial:

<https://github.com/Lebot007/gqs-algoritmo-03-py>

Adotou-se o modelo de ramificação **GitHub Flow**, ideal para entregas contínuas e projetos ágeis. A branch principal `main` é protegida e permanece sempre em estado estável de produção. O desenvolvimento de novas funcionalidades, ajustes e documentação ocorre em branches temporárias derivadas da `main`:

- `feature/motor-simulacao-vagas` — Implementação do loop `requestAnimationFrame`.
- `feature/supabase-rls-auth` — Configuração do SDK, logins e regras RLS no PostgreSQL.
- `fix/ajuste-trigger-tempo` — Correção da função em PL/pgSQL para cálculo exato em minutos.
- `docs/especificacao-tecnica` — Elaboração do documento técnico e relatório executivo.

4.2 Convenção de Commits (Conventional Commits)

Para garantir um histórico auditável, limpo e legível, a equipe estabeleceu a especificação *Conventional Commits*:

- `feat(simulacao):` adiciona suporte a aceleração multivelocidade 1x 2x e 3x

- `fix(banco)`: corrige filtro de sessão em buscas na tabela `movimentacoes`
- `docs(readme)`: insere instruções completas de execução local e estrutura de arquivos
- `refactor(ui)`: simplifica chamadas do Web Animations API na entrada de carros

4.3 Política de Pull Requests e Code Review

Nenhum código pode ser mesclado diretamente na branch `main` sem passar pelo processo formal de **Pull Request (PR)**. Os critérios obrigatórios para aceite do PR são:

1. **Aprovação Obrigatória:** Pelo menos 1 integrante da equipe diferente do autor do PR (João Vitor, Rafael Luiz ou Pietro Cardoso) deve revisar e aprovar as alterações.
2. **Ausência de Regressões:** Execução manual dos cenários de teste de QA para validar que o motor de simulação e a gravação no banco continuam operacionais.
3. **Inspecção de Console:** Verificação de que não há erros de sintaxe ou warnings não tratados no DevTools do navegador.

4.4 Documentação do Repositório (README.md)

O arquivo `README.md` na raiz do repositório disponibiliza um guia detalhado contendo a arquitetura de arquivos do projeto, links para CDNs externas (Supabase e jsPDF), instruções para provisionamento da tabela e trigger no Supabase Dashboard, e passos para execução local através de qualquer servidor HTTP de arquivos estáticos (como *Live Server* do VS Code ou `npx serve`).

5. Plano de Garantia de Qualidade (QA)

O Plano de Garantia de Qualidade definiu cenários de teste de integração fim a fim (E2E) para validar os fluxos mais críticos da aplicação.

5.1 Matriz de Cenários de Testes Integrados

ID	CENÁRIO DE TESTE	ENTRADA / PRÉ-CONDIÇÃO	PASSO A PASSO DE EXECUÇÃO	RESULTADO ESPERADO
CT01	Validação de Entrada, Saída e Cálculo Automático do Tempo via Trigger PL/pgSQL.	Aplicação carregada, simulação em execução e usuário logado como Admin no painel.	1. Observar veículo entrar no pátio (ex: Vaga 02, Placa ABC1D23). 2. Checar inclusão de linha com status <code>ESTACIONADO</code> e <code>tempo_minutos = null</code> . 3. Aguardar ciclo de saída do veículo e o disparo do <code>UPDATE</code> .	O status altera para <code>SAIU</code> , a <code>hora_saida</code> é registrada e a trigger <code>trg_calcular_tempo</code> preenche <code>tempo_minutos</code> automaticamente com um número inteiro ≥ 1 .

ID	CENÁRIO DE TESTE	ENTRADA / PRÉ-CONDIÇÃO	PASSO A PASSO DE EXECUÇÃO	RESULTADO ESPERADO
CT02	Bloqueio por RLS sem Login e Liberação de Leitura com Autenticação Admin.	Aplicação recarregada, sessão limpa, usuário não autenticado (Visitante).	1. Acessar a aplicação e observar o Painel Administrativo. 2. Verificar presença do card bloqueado 🔒. 3. Clicar em "Admin", inserir e-mail e senha válidos do Supabase Auth e clicar em "Entrar".	Visitante não consegue visualizar os dados (consulta <code>SELECT</code> bloqueada por RLS). Após o login bem-sucedido, o painel é desbloqueado e os dados da sessão são exibidos.
CT03	Limpeza Total do Banco ("Reiniciar" / F5) e Geração de PDF Isolado por Sessão.	Simulação rodando na Sessão 1 com mais de 10 registros na tabela.	1. Clicar em "Reiniciar" ou pressionar F5 (recarregar a página). 2. Iniciar nova simulação (Sessão 2) e registrar 5 veículos. 3. Clicar no botão "Exportar PDF".	A ação de reiniciar/recarregar executa <code>DELETE</code> na tabela <code>movimentacoes</code> , limpando todos os registros antigos. O PDF gerado reflete exclusivamente os dados da Sessão 2.
CT04	Comutação de Velocidades (1x, 2x, 3x) no Motor Visual e no Relógio.	Simulação iniciada rodando em velocidade padrão (1x).	1. Observar velocidade do timer e animação. 2. Clicar nos botões "2x" e "3x". 3. Retornar o seletor para "1x".	A velocidade do timer acelera proporcionalmente (2x e 3x), o tempo entre entrada/saída de carros diminui e a variável CSS <code>--pista-vel</code> ajusta o ritmo das pistas e carros sem travar a UI.

6. Bônus: Roteiro da Apresentação Técnica

Abaixo é apresentado o roteiro estruturado em 9 slides executivos para apresentação do pitch técnico para a banca e professores:

SLIDE	TÍTULO DO SLIDE	CONTEÚDO VISUAL E TÓPICOS DE APRESENTAÇÃO (BULLET POINTS)
Slide 1	Capa da Apresentação	- Projeto: ParkSim — Sistema Simulador de Estacionamento Inteligente. - Integrantes: João Vitor, Rafael Luiz Ferreira de Souza, Pietro Cardoso de Oliveira. - Disciplina: Gestão e Qualidade de Software Prof. Daniel Henrique Matos de Paiva UNA Barreiro (2026).
Slide 2	Problema e Escopo do Produto	- Desafio: Ausência de visibilidade em tempo real do fluxo de vagas e sobrecarga do client-side na gestão de regras de negócio. - Solução ParkSim: Simulador web interativo para 8 vagas, sem frameworks, integrado a banco relacional em nuvem.
Slide 3	Requisitos Funcionais e Fluxo Crítico	- Fluxo de Vida: Gerador de placa Mercosul + Cor -> Animação de Entrada -> Registro em Banco -> Permanência -> Saída Animada -> Atualização de Banco. - Destaques RF: Controles 1x/2x/3x, Painel de relatórios ao vivo e Exportação de PDF com conceitos didáticos.

SLIDE	TÍTULO DO SLIDE	CONTEÚDO VISUAL E TÓPICOS DE APRESENTAÇÃO (BULLET POINTS)
Slide 4	Metas de Qualidade (ISO/IEC 25010)	- Desempenho: Renderização a 60 FPS com transform/opacity e Web Animations API. - Segurança: RLS no PostgreSQL (SELECT restrito a administradores JWT). - Conformidade LGPD: Apenas dados simulados sem coleta de PII.
Slide 5	Arquitetura de Dados (DER e SQL)	- Tabela <code>movimentacoes</code> com chaves primárias, carimbos de tempo <code>TIMESTAMPTZ</code> e controle de estado. - Regra no Banco: Trigger <code>trg_calcular_tempo</code> acionada <code>BEFORE UPDATE</code> para cálculo automatizado de minutos.
Slide 6	Diagrama de Classes / Módulos JS	- Arquitetura Vanilla ES Modules: 6 módulos desacoplados (<code>config.js</code> , <code>supabase.js</code> , <code>banco.js</code> , <code>simulacao.js</code> , <code>ui.js</code> , <code>app.js</code>). - Motor de Ticks baseado em <code>requestAnimationFrame</code> .
Slide 7	Pipeline Git e Code Review	- Modelo GitHub Flow com branch <code>main</code> protegida. - Padrão <i>Conventional Commits</i> (feat, fix, docs). - Code Review obrigatório por ao menos 1 par antes do merge em produção.
Slide 8	Garantia de Qualidade e Testes (QA)	- Apresentação dos 4 Cenários de Testes (CT01 ao CT04). - Validação prática de Triggers, Políticas RLS, Expurgos de Banco e Seletor de Velocidades.
Slide 9	Conclusão e Desafios Identificados	- Principais Aprendizados: Sincronismo entre física de animações client-side e banco BaaS assíncrono. - Conclusão: Entrega completa dentro das normas da Lista de Exercícios III com alta qualidade técnica.